

Exam Solution for TDT4186-resit

August 2024

Notes for grading:

- The total amounts to **50 points**. Grades are calculated as follows:

A	44- 50
B	38 - 43
C	32 - 27
D	26 - 31
E	20 - 25
F	0 - 19

1 Multi-Choice Questions

Automatic grading.

1. Monolithic kernel OSs have all system services and functionalities in kernel space.
2. `fork()`. Note that not all OSs use `fork()` to create processes.
3. The program counter of a process is used to indicate the next instruction to be executed by the process.
4. T1 and T2 share the stack.
5. First come first serve (FCFS)
6. 2,3
7. 1,2,3
8. Segmentation fault. The bound register determines the address range. 500 is converted to 0x1F4 in hexadecimal. The memory range should be in $[0x0A10, 0X0C04]$ $0x0A10 + 0x0230 = 0x0C40$ is out of the range, so it should prompt a segmentation fault.
9. Least-recently-used
10. page number

11. m-n, n
12. 1,3
13. mutual exclusive
14. either mutex locks or binary semaphores. A binary semaphore can be used to implement a multex lock.
15. 2,3.
16. The average rotation latency of this HDD is 5.5ms.
 - $\frac{60s}{5400} = 0.011s = 11ms$. The average rotation latency is 5.5ms.
 - The latency to read 1MB file: $5.5ms + 8ms + \frac{1MB}{100MB/s} \times 1000ms = 23.5ms$
17. SSDs use flash memory to store data.
18. Two different files can have the same Inode number at the same time.

2 Multi-part questions

For multi-part questions, the automatic grading may fail to award some answers which are slightly different from the provided solutions or written in different phrases. Thus, it is worth noting that all wrong blanks should go through a scrutiny check. Answers that provide a close meaning can be awarded the point without deduction.

Process

Explanation: `fork()==0` indicates that it is executing the child process, which will be used to execute the new command. Thus

- `exec();`
- `wait();`
- 1. Only the parent will print out "Shell%".

Scheduling

Explanation: See the results from Figure. 1 and 2.

- 25
- 12.25. 12.2 and 12 are also correct.
- 13.5

Job	Arrival Time	Burst Time	Finish Time	Turnaround Time	Waiting Time
A	0	20	51	51	31
B	3	16	34	31	15
C	9	9	18	9	0
D	15	6	24	9	3
Average				$100 / 4 = 25$	$49 / 4 = 12.25$

Figure 1: Results of STCF

Job	Arrival Time	Burst Time	Finish Time	Turnaround Time	Waiting Time
A	0	20	20	20	0
B	3	16	51	48	32
C	9	9	35	26	17
D	15	6	26	11	5
Average				$105 / 4 = 26.25$	$54 / 4 = 13.5$

Figure 2: Results of SJF

Memory

Explanation: The total is **6 points**.

- Segmentation and paging. **Each one accounts for 1 point.**
- Segmentation divides an address space into several logical segments of varying size and allocates them to different locations in memory; Paging, on the other hand, divides the address space into many fixed-size pages, and physical memory is also divided into fixed-size frames. Pages can be flexibly assigned to any free frames in memory. **The correct answer should convey a similar meaning for both parts. Each part accounts for 1 point.**
- External fragmentation for segmentation and internal fragmentation for paging. **The correct answer should clearly specify the issue for each method. Each part accounts for 1 point.**

Page

Explanation: The total is **5 points**. Each blank account for **1 point**.

- 11. $2KB = 2^{11}$, so 11 bits are needed.
- 512 or 2^9 . Each page is 2KB, and each page entry is 4B. $2^{11}/2^2 = 512$ or 2^9 .
- 2^{27} . $2^{38} \div 2^{11} = 2^{27}$
- 2^{38} or 256 GB
- 9. 27 bits for page index, and each page can store 2^9 page table entries. Additionally, three levels have the same size.

Concurrency

Explanation: The solution should be free of race conditions and deadlocks. Two points for initialization, two points for producer, and two points for consumer.

Initialization:

- `sem.t empty = M`
- `sem.t full = 0`
- `sem.t mutex = 1`

Producer:

- `sem_wait(empty)`
- `sem_wait(mutex)`
- `sem_post(mutex)`

- sem_post(full)

Consumer:

- sem_wait(full)
- sem_wait(mutex)
- sem_post(mutex)
- sem_post(empty)

I/O Devices

- Polling continuously checks the status of an I/O device by reading its status register; I/O devices use interrupt signals to communicate with CPU. Instead of repeatedly checking the status of an I/O device, I/O interrupts wait for an interrupt signal to be sent to CPU, which wakes up the thread/process that issued the I/O request.
- Polling: inefficient use of CPU time; Interrupts: high context switching overhead, not good for fast devices
- Hybrid approach: it polls for a short time, and if the device has not yet finished, it then uses interrupts.

File System

Explanation: The total is **4 points**, and each blank accounts for **1 point**.

- read
- write
- read
- write